

03

MÓDULO

DISCIPLINA · INTRODUÇÃO À PROGRAMAÇÃO ESTRUTURADA EM PYTHON

Estrutura Condicional em Python

`if · elif · else` ● `switch case` ● `break / continue`

Agenda do Módulo

4 SEÇÕES • ~2 AULAS DE 50 MIN

01

FUNDAMENTOS

Fundamentos

Conceito de tomada de decisão em programas e os operadores que sustentam as condições.

- 0 que são estruturas condicionais
- Operadores relacionais e lógicos
- Fluxograma de decisão

02

SINTAXE

if · elif · else

A família principal de condicionais do Python, com sintaxe, exemplos e armadilhas.

- if simples e if-else
- if-elif-else em cadeia
- Pegadinhas comuns

03

PATTERN MATCHING

Switch Case & Match

Comparação entre switch de outras linguagens e o pattern matching do Python 3.10+.

- Switch em C/Java/JS
- Por que Python não tinha
- Match-case moderno

04

CONTROLE DE FLUXO

break / continue

Controle de fluxo dentro de loops: interromper o laço ou pular uma iteração.

- break – sair do loop
- continue – pular iteração
- Boas práticas

01



SEÇÃO 01

Fundamentos das Estruturas Condicionais

Antes de escrever um único **if**, precisamos entender o que significa tomar uma decisão dentro de um programa — e quais operadores sustentam essa decisão.

- O que são estruturas condicionais
- Operadores relacionais e lógicos
- Fluxograma de tomada de decisão

CONCEITO

O que são estruturas condicionais

Estruturas condicionais são comandos que permitem ao programa executar blocos diferentes de código dependendo de uma expressão avaliada como **verdadeira (True)** ou **falsa (False)**. Elas são o mecanismo fundamental pelo qual um programa deixa de ser uma sequência rígida de instruções e passa a **tomar decisões**.

Em Python, toda condição é avaliada como um valor booleano. O interpretador pergunta *"isso é verdadeiro?"* e, conforme a resposta, escolhe qual bloco executar — podendo **pular, repetir ou desviar** o fluxo.

- ▶ Validar se uma senha tem o tamanho mínimo
- ▶ Decidir se um aluno foi aprovado ou reprovado
- ▶ Mostrar telas diferentes conforme o tipo de usuário

🏠 Analogia do dia a dia

"Se está chovendo, levanto o guarda-chuva. Senão, saio de mãos vazias." — Em programação, essa mesma estrutura se traduz em: avaliar uma condição (**está chovendo?**) e, conforme o resultado, executar uma ação (**abrir guarda-chuva**) ou outra (**sair sem**).

```
if chovendo: abrir_guarda_chuva() else: sair()
```

PYTHON

✦ True → executa o bloco do if · False → executa o else

Operadores: a base das condições

VOCABULÁRIO · 9 OPERADORES

OPERADORES RELACIONAIS			
OPERADOR	SIGNIFICADO	EXEMPLO	RESULTADO
=	Igual a	5 = 5	True
≠	Diferente de	5 ≠ 3	True
>	Maior que	7 > 4	True
<	Menor que	2 < 8	True
≥	Maior ou igual	5 ≥ 5	True
≤	Menor ou igual	4 ≤ 3	False

OPERADORES LÓGICOS			
OPERADOR	SIGNIFICADO	EXEMPLO	RESULTADO
and	E lógico (ambos)	True and False	False
or	Ou lógico (ao menos um)	True or False	True
not	Negação (inverte)	not True	False
⚡ Short-circuit: em A and B, se A for falso, B nem é avaliado. Precedência: not > and > or.			

**Dica:** Combine operadores com parênteses para evitar ambiguidade: `(idade ≥ 18) and (tem_cnh)` é mais legível e seguro do que `idade ≥ 18 and tem_cnh`.

```
if (idade ≥ 18) and tem_cnh: print("Pode dirigir")
```

EXEMPLO · PYTHON

VISUALIZAÇÃO

Fluxograma de uma decisão

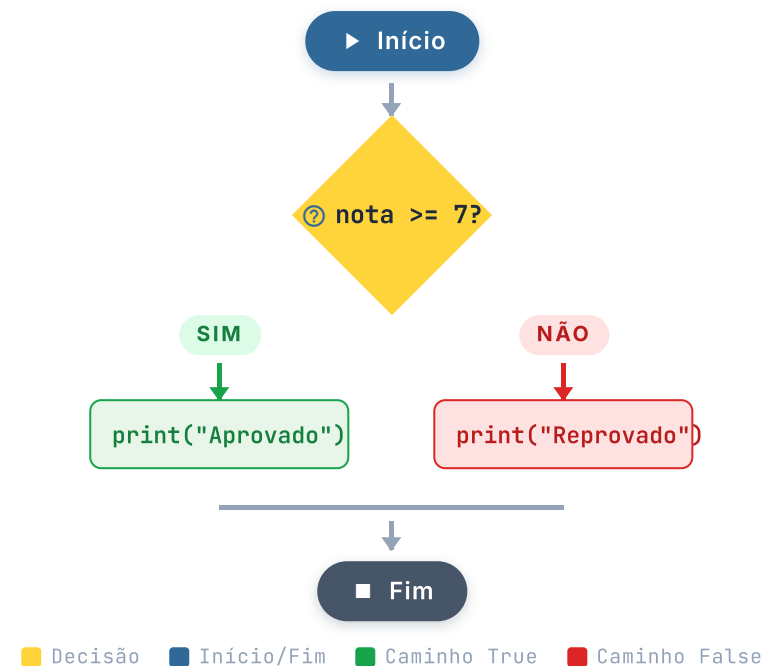
Um fluxograma representa graficamente o caminho que o programa percorre. O **losango** indica um ponto de decisão (uma condição); as setas mostram os dois caminhos possíveis — **verdadeiro (sim)** e **falso (não)**. Cada caminho leva a um bloco de ação diferente.

```
if nota ≥ 7: print("Aprovado") else:  
    print("Reprovado")
```

PYTHON

FLUXOGRAMA

if-else



02



SEÇÃO 02

A família `if` / `elif` / `else`

A estrutura condicional mais utilizada em Python. Aprenderemos a sintaxe, as variações (simples, binária e em cadeia) e as armadilhas que mais derrubam iniciantes.

- `if simples` — executa ou pula
- `if-else` — escolha binária
- `if-elif-else` — múltiplos caminhos
- Pegadinhas comuns do `if`

SINTAXE

if simples

O **if** mais básico avalia uma condição. Se for verdadeira, executa o bloco indentado. Se for falsa, o programa simplesmente pula o bloco e segue.

```
if <condicao>:  
    <comando 1>  
    <comando 2>  
    <comando seguinte>
```

SINTAXE

- **palavra-chave** — inicia a condicional
- **dois pontos obrigatórios** — abrem o bloco
- **indentação de 4 espaços** — pertence ao if

EXEMPLO PRÁTICO

Verificação de idade

```
idade = int(input("Sua idade: "))  
if idade ≥ 18:  
    print("Maior de idade")  
    print("Pode tirar CNH")  
print("Fim do programa")
```

EXEMPLO.PY

⚠ **Regra:** o bloco pertence ao **if** enquanto estiver indentado. Qualquer linha alinhada com o if (sem recuo) está **FORA** da condicional.

DECISÃO BINÁRIA

if / else

Quando há exatamente dois caminhos possíveis, usamos **if** acompanhado de **else**. Se a condição for verdadeira, executa o bloco do **if**; caso contrário, executa o bloco do **else**. Não há como escapar: um dos dois sempre roda.

```
if <condicao>:
    # executado se verdadeiro
    <comandos>
else:
    # executado se falso
    <comandos>
```

SINTAXE



O **else** **NÃO** recebe condição. Ele é o caminho padrão para tudo que não entrou no **if**.

EXEMPLO PRÁTICO

Par ou ímpar

```
numero = int(input("Digite um inteiro: "))
if numero % 2 == 0:
    print(f"{numero} é PAR")
else:
    print(f"{numero} é ÍMPAR")
```

PARIDADE.PY

ENTRADA 8

> 8 é PAR

ENTRADA 7

> 7 é ÍMPAR

MÚLTIPLAS CONDIÇÕES

if / elif / else

Quando há três ou mais caminhos, encadeamos com **elif** (contração de `else if`). O Python avalia as condições de cima para baixo e executa **APENAS** o primeiro bloco cuja condição seja verdadeira. As demais não são testadas.

```
if <cond1>:
    <bloco 1>
elif <cond2>:
    <bloco 2>
elif <cond3>:
    <bloco 3>
else:
    <bloco padrão>
```

SINTAXE

! **Atenção à ordem!** Coloque sempre as condições mais específicas/restritivas primeiro. Se começar pela mais genérica, ela engolirá todas as outras.

EXEMPLO PRÁTICO

Classificação do IMC

```
imc = 22.5
if imc < 18.5:
    print("Abaixo do peso")
elif imc < 25:
    print("Peso normal")
elif imc < 30:
    print("Sobrepeso")
else:
    print("Obesidade")
```

IMC.PY

SAÍDAS ESPERADAS

ENTRADA	SAÍDA
imc = 17	Abaixo do peso
imc = 22	Peso normal
imc = 27	Sobrepeso
imc = 32	Obesidade

ESTUDO DE CASO

Cálculo da situação do aluno

Programa que recebe duas notas, calcula a média e decide a situação acadêmica segundo o regulamento da instituição.

```
SITUACAO_ALUNO.PY
# Programa: situação do aluno
nota1 = float(input("Nota 1: "))
nota2 = float(input("Nota 2: "))
media = (nota1 + nota2) / 2

if media >= 7.0:
    situacao = "Aprovado"
elif media >= 5.0:
    situacao = "Recuperação"
else:
    situacao = "Reprovado"

print(f"Média: {media:.1f} – Situação: {situacao}")
```

Observação: a regra "média ≥ 7 = aprovado" e "média ≥ 5 = recuperação" é apenas um exemplo. Cada instituição tem seu critério — sempre confirme o regulamento vigente.

TESTES

Trace a saída para cada entrada

NOTA 1	NOTA 2	MÉDIA	SAÍDA
8.0	9.0	8.5	Aprovado
6.0	7.0	6.5	Recuperação
4.0	3.0	3.5	Reprovado
7.0	7.0	7.0	Aprovado
5.0	5.0	5.0	Recuperação

● Aprovado ● Recuperação ● Reprovado

Pegadinhas comuns do if

4 ERROS RECORRENTES · EVITE DESDE JÁ

1 · = vs ==

```
✗ if x = 5:           # SyntaxError

✓ if x == 5:         # comparação correta
```

= atribui, == compara. Em Python o erro é capturado em tempo de compilação; em C/Java seria um bug silencioso.

2 · Esquecer os dois pontos

```
✗ if idade ≥ 18

    print(...)

✓ if idade ≥ 18:

    print(...)
```

O : é obrigatório ao final de if, elif, else, for, while, def, class.

3 · Indentação inconsistente

```
✗ if ok:

    a()

    b()           # IndentationError

✓ if ok:

    a()

    b()
```

4 · Comparar floats com ==

```
✗ if 0.1 + 0.2 == 0.3:  # False!

✓ if abs(0.1+0.2-0.3) < 1e-9:
```

Floats têm imprecisão binária. Compare com tolerância (epsilon) ou use round().

03

SEÇÃO 03

Switch Case e o Match do Python

Outras linguagens têm **switch** desde os anos 70. O Python demorou — mas quando chegou, chegou mais forte. Conheça o histórico e o **pattern matching** moderno.

- 01 Switch em C, Java e JavaScript
- 02 Por que Python não tinha switch
- 03 Match-case (Python 3.10+)

Switch: como era em outras linguagens

3 ABORDAGENS · MESMA LÓGICA

TRADICIONAL

C / Java / JS

```
switch (dia) { case 1: print("Segunda"); break;
case 2: print("Terça"); break; default:
print("Outro"); }
```

Precisa de **break** em cada case. Esquecer = bug de **fall-through**.

WORKAROUND

Python (antes de 3.10)

```
if dia == 1: print("Segunda") elif dia == 2:
print("Terça") else: print("Outro")
```

Funciona, mas verboso. Repete a variável dia a cada linha.

MODERNO

Python 3.10+ (match)

```
match dia: case 1: print("Segunda") case 2:
print("Terça") case _: print("Outro")
```

Sem **fall-through**, sem **break**. `_` é o coringa (default).

↓ **Fall-through:** C/Java têm (precisa break) · Python match **NÃO** tem (cada case é exclusivo).

🚩 **Default:** palavra-chave default · Python usa `_` (coringa).

✦ **Poder:** switch só compara igualdade · match faz **pattern matching estrutural**.

PATTERN MATCHING

Match-case

O **match-case** avalia uma expressão e tenta casá-la com cada padrão (case). O primeiro padrão que encaixar é executado; os demais são ignorados. O `_` (underscore) é o coringa que casa com qualquer valor — equivalente ao *default* do switch.

- ▶ **Sem fall-through** — cada case é exclusivo
- ▶ **Sem break implícito** necessário
- ▶ Suporta **padrões estruturais** (tuplas, listas, dicts)
- ▶ **Captura variáveis** (binding)

Disponível a partir do Python 3.10 (out/2021). Verifique a versão com `python --version`

Exemplo: roteamento de comandos

```
comando = input("> ").strip().split()

match comando:
    case ["sair"]:
        print("Encerrando...")
    case ["ajuda"]:
        print("Comandos: sair, ajuda, olá <nome>")
    case ["olá", nome]:
        print(f"Olá, {nome}!")
    case ["somar", a, b]:
        print(int(a) + int(b))
    case _:
        print("Comando não reconhecido")
```

ENTRADA	SAÍDA
'sair'	Encerrando...
'olá Maria'	Olá, Maria!
'somar 3 4'	7
'pular'	Comando não reconhecido

04

SEÇÃO 04

Break e Continue — controle de loops

break e **continue** quase sempre vivem dentro de um **if** — é ele que decide **QUANDO** interromper ou pular. E o **break** obrigatório do **switch** tradicional simplesmente desaparece no **match-case** do Python.

- **break** — dentro de **if**, em loops
- **continue** — dentro de **if**, em loops
- **break** no **switch** vs **match-case**
- Boas práticas e armadilhas

INTERROMPER

break

O **break** encerra imediatamente o laço (for ou while) mais interno. Quase nunca é usado solto — ele vive dentro de um **if**, que decide **QUANDO** o laço deve ser interrompido. O **if** é o gatilho; o loop é o alvo.

```
for item in colecao:
    if condicao:           # if decide QUANDO
        break            # loop é encerrado
# fluxo continua aqui
```

SINTAXE

if decide **QUANDO**; o loop decide **ONDE**. Sem loop em volta, **break** dentro de **if** gera **SyntaxError** — break é controle de laço, não de if.

🔍 BREAK DENTRO DE IF (DENTRO DE LOOP)

```
numeros = [3, 7, 2, 9, 5, 1]
alvo = 9
for n in numeros:
    print(f"Verificando {n}...")
    if n == alvo:           # if dispara o break
        print(f"Achei {alvo}!")
        break
```

SAÍDA ESPERADA

```
> Verificando 3...
> Verificando 7...
> Verificando 2...
> Verificando 9...
> Achei 9!
```

➔ BREAK NO SWITCH TRADICIONAL VS MATCH-CASE

C / JAVA / JS – TRADICIONAL

```
switch (op) {
    case 1:
        fazA();
        break;           // obrigatório!
    case 2:
        fazB();
        break;           // obrigatório!
}
```

PYTHON 3.10+ – MODERNO

```
match op:
    case 1:
        fazA()           # sem break
    case 2:
        fazB()           # sem break
```

Em C/Java/JS, o **break** evita o **fall-through** (continuar no próximo case). No **match-case** do Python, cada case é exclusivo por padrão — break é desnecessário e gera **SyntaxError** se usado.

PULAR

continue

O **continue** interrompe a iteração atual e salta para a próxima. Assim como o **break**, ele vive dentro de um **if** — que decide **QUANDO** pular. **continue** é exclusivo de loops (for/while): **NÃO** existe **continue** em **if** isolado nem em **match-case**.

```
for item in colecao:
    if condicao:           # if decide QUANDO
        continue         # pula para o próximo
                        # restante do bloco
```

Padrão clássico: **if** + **continue** no início do loop para filtrar casos que não devem ser processados. O **if** é o filtro; o **continue** é a ação.

continue **NÃO** existe em **match-case**. E fora de um loop, "**if cond: continue**" é **SyntaxError**.

EXEMPLO: SOMAR APENAS OS PARES

```
numeros = [1, 2, 3, 4, 5, 6, 7, 8]
soma = 0
for n in numeros:
    if n % 2 != 0:           # if filtra ímpares
        continue           # pula direto
    soma += n
    print(f"Somei {n}, parcial = {soma}")
print(f"Total: {soma}")
```

```
SAÍDA ESPERADA
> Somei 2, parcial = 2
> Somei 4, parcial = 6
> Somei 6, parcial = 12
> Somei 8, parcial = 20
> Total: 20
```

COMANDO	ONDE FUNCIONA	PAPEL DO IF
break	for / while (e switch em C)	Gatilho — decide QUANDO
continue	for / while apenas	Filtro — decide O QUE pular

Nenhum dos dois funciona em **match-case**. E nenhum dos dois funciona sem um loop em volta.

ESTUDO DE CASO

Validação de senha com `if` + `break/continue`

Programa que pede a senha até 3 vezes. Usa `continue` (dentro de `if`) para repetir entradas vazias, e `break` (dentro de `if`) para sair ao acertar.

```
senha_correta = "python123"
tentativas = 0
while tentativas < 3:
    entrada = input("Senha: ")
    if entrada == "":           # if → gatilho
        print("Vazia, tente novamente")
        continue              # pula iteração
    tentativas += 1
    if entrada == senha_correta: # if → gatilho
        print("Acesso concedido!")
        break                 # sai do while
    print("Senha incorreta")
if tentativas == 3:
    print("Conta bloqueada")
```

SENHA.PY

Cada `break/continue` está dentro de um `if`. O `if` decide a **CONDIÇÃO**; o loop sofre a **AÇÃO**. Esse é o padrão correto — nunca use `break/continue` solto.

⚠ PEGADINHAS COMUNS

**break/continue precisam de loop**

Fora de `for/while`, "`if cond: break`" gera `SyntaxError`. `break` e `continue` são controles de LAÇO, não de `if` isolado.

**break no match-case é proibido**

Em Python, `match-case` não usa `break` — cada case é exclusivo por padrão. Colocar `break` gera `SyntaxError`.

**break só sai do laço interno**

Em loops aninhados, `break` encerra apenas o `for/while` mais interno. O externo continua normalmente.

**continue no while vira loop infinito**

Se o incremento estiver **DEPOIS** do `continue`, o contador nunca avança. Sempre incremente **ANTES** do `continue`, ou use `for`.

SÍNTESE

O que vimos neste módulo



Fundamentos

Tomada de decisão, operadores relacionais e lógicos, fluxogramas.



if · elif · else

Condicional simples, binária e em cadeia. Sintaxe, exemplos e pegadinhas.



Switch & Match

Histórico em outras linguagens e o pattern matching do Python 3.10+.



break & continue

Controle fino de loops: interromper ou pular iterações com responsabilidade.

Próximos passos

🏠 PARA CASA

- ▶ Reescrever 3 programas anteriores usando if-elif-else
- ▶ Implementar uma calculadora com match-case
- ▶ Resolver a lista de exercícios do Módulo 3 (8 questões)

→ PRÓXIMO MÓDULO

- ▶ **Módulo 4:** Estruturas de Repetição (for, while)
- ▶ Aprofundaremos loops onde break e continue vivem
- ▶ Conectaremos condicionais + repetições em programas completos